F2 · DAY6 · WEEK1 MOIS 1 · SEMAINE 1 · JOUR 6 · FICHE 2 SYNTHÈSE

Synthèse de la semaine 1

Tout ce qu'il faut savoir à la fin de la semaine « Python & Developer Foundations », sur quelques pages. Chaque thème renvoie à la fiche détaillée. À relire avant chaque évaluation.

1. Variables et types
2. Conditions
3. Boucles et comptage
4. Structures de données
5. Références et copies
6. Fonctions
7. Fichiers et JSON
8. Erreurs
9. Modules et outils
10. Terminal
11. Git et qualité
12. Les 12 pièges de la semaine

1. Variables et types · F1.DAY2

RÈGLE	EXEMPLE
`=` range, `==` compare	`x = 5` · `x == 5` → `True`
On calcule à droite, puis on range à gauche	`x = x * 3`
`int` · `float` · `str` · `bool`	`12` · `0.92` · `"chat"` · `True`
Des guillemets, c'est toujours du texte	`type("3.5")` → `str`
Convertir avant de calculer	`int("18") + 2` → `20`
`input()` renvoie toujours du texte	`float(input("Note ? "))`

2. Conditions · F1.DAY2

```
if note >= 16:          # SI
    print("Très bien")
elif note >= 10:        # SINON, SI  → même bloc : un seul gagnant
    print("Admis")
else:                   # SINON
    print("Ajourné")
```

Deux-points à la fin, 4 espaces dessous. Python s'arrête à la première condition vraie. Un nouveau `if` ouvre un nouveau bloc, testé quoi qu'il arrive. `>` est strict, `>=` inclut l'égalité.

3. Boucles et comptage · F1.DAY2, F2.DAY1

```
compteur = 0                    # la tirelire part de 0
for note in notes:              # pour chaque note
    if note >= 10:
        compteur = compteur + 1  # on AJOUTE (et non compteur = 1)
print(compteur)                 # après la boucle : affiché une seule fois
```

OUTIL	USAGE
`range(n)`	0, 1, …, n−1
`enumerate(seq, start=1)`	(numéro, élément)
`zip(a, b)`	deux listes en parallèle
`[x for x in seq if cond]`	une nouvelle liste filtrée
`Counter(seq).most_common(k)`	le top k des valeurs les plus fréquentes
`sorted(seq, key=…, reverse=True)`	une nouvelle liste triée

4. Structures de données · F1.DAY1

TYPE	SYNTAXE	MODIFIABLE	DOUBLONS	ON Y ACCÈDE PAR
list	`[1, 2]`	oui	oui	position `[0]`
tuple	`(1, 2)`	non	oui	position `[0]`
dict	`{"nom": "Lina"}`	oui	clés uniques	clé `["nom"]`
set	`{1, 2}`	oui	non	test `x in s`

5. Références et copies · F1.DAY1

RÈGLE	EXEMPLE
Une variable est une étiquette sur un objet	`b = a` : deux étiquettes, une seule liste
`.append()` modifie l'objet : toutes les étiquettes le voient	`b.append(3)` → `a` change
`=` déplace l'étiquette vers un nouvel objet	`a = [9]` → `b` ne change pas
Mutables : list, dict, set. Immuables : int, float, str, tuple, bool	
Vraie copie : `.copy()`, ou `deepcopy()` si les données sont imbriquées	`notes = notes.copy()`

6. Fonctions · F1.DAY3

```
def calculer_moyenne(notes):            # def fabrique, notes = paramètre
    """Renvoie la moyenne de notes."""
    return sum(notes) / len(notes)      # return donne ET arrête la fonction

m = calculer_moyenne([12, 16])          # l'appel : [12, 16] = argument
```

Sans `return`, une fonction renvoie `None`. `print` montre, `return` donne. Les variables créées dans une fonction y restent. Une fonction = une tâche = un verbe. Elle ne modifie pas ses entrées (pas d'effet de bord).

7. Fichiers et JSON · F1.DAY3

```
with open("notes.txt", encoding="utf-8") as f:     # se referme tout seul
    for ligne in f:
        ligne = ligne.strip()                       # c'est du texte : on netto
        nom, note = ligne.split(";")                # ...on découpe...
        note = float(note)                          # ...puis on convertit

with open("rapport.json", "w", encoding="utf-8") as f:
    json.dump(rapport, f, indent=2, ensure_ascii=False)   # dict → fichier
```

`"r"` lit, `"w"` efface puis écrit, `"a"` ajoute. `f.write()` n'écrit que du texte, sans `\n` automatique. `json.load(f)` fait le chemin inverse : fichier → dict.

8. Erreurs · F2.DAY3

ERREUR	CAUSE
`SyntaxError`	code mal écrit : `:`, parenthèse ou guillemet oublié (ne s'attrape pas)
`IndentationError`	des espaces au mauvais endroit
`NameError`	nom inconnu ou faute de frappe
`TypeError`	types incompatibles : `"18" + 2`
`ValueError`	valeur impossible : `float("abc")`
`IndexError` / `KeyError`	case ou clé inexistante
`FileNotFoundError`	fichier introuvable, souvent un chemin relatif lancé du mauvais dossier

```
try:
    note = float(texte)
except ValueError as e:      # le filet : on attrape l'erreur PRÉCISE
    rejets.append(texte)
```

Une erreur se lit de bas en haut : type, message, puis ligne. Une erreur dans `try` fait sauter la suite du `try`. `raise ValueError("…")` refuse une donnée impossible.

9. Modules et outils · F2.DAY2, F2.DAY4

NOTION	À RETENIR
`import`	exécute tout le fichier importé : on protège les tests par `if __name__ == "__main__":`
venv	une cuisine par projet : `py -m venv .venv`, puis `source .venv/Scripts/activate`
pip	`python -m pip install x`, `pip freeze > requirements.txt`, `pip install -r requirements.txt`
Arguments	`sys.argv[1]` est le premier mot après le nom du script, et c'est du texte
Chemins	`Path(__file__).parent` marche d'où que l'on lance le programme

10. Terminal · F1.DAY4

COMMANDE	EFFET	COMMANDE	EFFET
`pwd`	où suis-je	`cat` / `head` / `tail`	lire
`ls`	qu'y a-t-il ici	`wc -l`	compter les lignes
`cd` / `cd ..`	bouger / remonter	`find` / `grep`	trouver le fichier / la phrase
`mkdir -p` / `touch`	créer	`|`	enchaîner des commandes
`cp` / `mv` / `rm`	copier / déplacer / supprimer (sans corbeille)	`>` / `>>`	écraser / ajouter dans un fichier

11. Git et qualité · F1.DAY5, F2.DAY5

```
git status → git add . → git status → git commit -m "Ajoute …" → git push
```

Un commit ne contient que l'index. Un commit = une idée, avec un verbe. Le `.gitignore` contient `.venv`, `__pycache__`, les données et les secrets. Côté qualité : PEP 8, des noms explicites, des fonctions courtes avec docstring, des commentaires qui expliquent le pourquoi, un README en 6 sections, et des tests pytest (normal, limite, vide, invalide).

12. Les 12 pièges de la semaine

1. `if x = 5` au lieu de `if x == 5`.
2. Oublier les `:` après `if`, `elif`, `else`, `for` et `def`.
3. Calculer avec du texte : `"18" + 2`.
4. `c = 1` au lieu de `c = c + 1` pour compter.

5	Confondre `elif` (même bloc) et un nouveau `if` (nouveau bloc).
6	`print` au lieu de `return` : la fonction renvoie `None`.
7	Croire que `b = a` copie une liste.
8	Une fonction qui modifie la liste qu'on lui donne (effet de bord).
9	Écrire une commande du terminal dans un fichier `.py`.
10	Exécuter sans avoir enregistré (le ● sur l'onglet).
11	Lancer depuis le mauvais dossier : `FileNotFoundError`.
12	Oublier `git add` avant `git commit`.
